

A Gentle Introduction to Continuous Optimization

Saurabh, Adjunct Faculty, WILP, BITS-Pilani

July 21, 2025

Contents

1	What is Optimization?	3
2	The Core Idea: Gradient Descent	3
2.1	Why the Gradient is the Direction of Fastest Ascent	4
2.2	The Update Rule	5
3	Epochs, Iterations, and Batches Explained	5
4	Gradient Descent Variants: A Detailed Numerical Example	5
4.1	1. Batch Gradient Descent (The Whole Dataset)	5
4.2	2. Mini-Batch Gradient Descent (A Small Batch)	6
4.3	3. Stochastic Gradient Descent (A Single Point)	7
4.4	Summary of Variants	8
5	The Learning Rate (α): The Goldilocks Problem	8

1 What is Optimization?

At its heart, optimization is the process of finding the **best** possible solution to a problem from a set of available alternatives. In mathematics and machine learning, this usually means finding the **minimum** or **maximum** value of a function, often called the **objective function**.

Key Idea: Objective Function

The objective function is the quantity you want to optimize. Think of it as a score. Our goal is to get the best score possible.

- **Minimizing Cost:** A company wants to find production levels that result in the lowest possible cost. The “score” to minimize is the cost.
- **Maximizing Profit:** A business wants to set prices to achieve the highest possible profit. The “score” to maximize is the profit.
- **Minimizing Error:** In machine learning, we want our model’s predictions to be as close to the real data as possible. We define a **loss function** that measures the model’s “unhappiness” or error. Our goal is to make the model as “happy” as possible by finding the parameters that result in the lowest loss.

There are two main flavors of continuous optimization:

1. **Unconstrained Optimization:** We are free to choose any value for our variables to find the minimum. There are no boundaries limiting our search.
2. **Constrained Optimization:** Our search for the minimum is limited by certain rules. The solution must satisfy these conditions.

Analogy: Finding the Lowest Point

Imagine you are trying to find the lowest point in a landscape.

- **Unconstrained:** You are in an open, rolling field and can walk anywhere. Your goal is to find the absolute lowest point in that field.
- **Constrained:** You are on a hiking trail on a mountain. You must stay on the path. Your goal is to find the lowest point *that is on the trail*, which might not be the absolute lowest point in the entire mountain range.

This guide focuses on **unconstrained optimization**.

2 The Core Idea: Gradient Descent

For simple functions, we can use calculus: find the derivative, set it to zero, and solve. But for complex functions, solving analytically is often impossible. This is where **Gradient Descent** comes in.

Analogy: The Foggy Mountain

Imagine you are on a foggy mountain and want to get to the lowest valley. You can't see the whole landscape, but you can see the ground at your feet. What's your strategy?

1. Look at the ground and find the direction of the **steepest downhill slope**.
2. Take a small **step** in that direction.
3. **Repeat** this process.

By repeating these steps, you will eventually walk down to a valley (a local minimum). Gradient Descent is the mathematical equivalent of this process.

2.1 Why the Gradient is the Direction of Fastest Ascent

The **gradient** of a function f , denoted ∇f , is a vector that points in the direction of the greatest rate of increase. Think of it as a magical compass that always points to the steepest "uphill" path from where you are standing. To understand why, we look at the **directional derivative**, $D_{\mathbf{u}}f$, which tells us the rate of change of f in the direction of any unit vector $\mathbf{u}$. It is defined as:

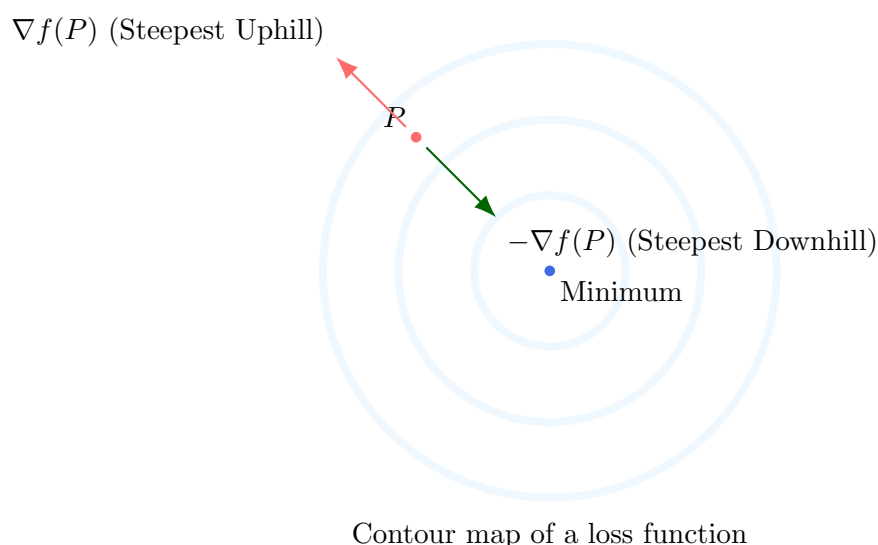
$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \|\mathbf{u}\| \cos(\theta) \quad (1)$$

where θ is the angle between the gradient vector ∇f and the direction vector $\mathbf{u}$. Since $\mathbf{u}$ is a unit vector, $\|\mathbf{u}\| = 1$. So, $D_{\mathbf{u}}f = \|\nabla f\| \cos(\theta)$.

To maximize this rate of change, we need to maximize $\cos(\theta)$. The maximum value of $\cos(\theta)$ is 1, which occurs when $\theta = 0$. This means the direction vector $\mathbf{u}$ must point in the **same direction** as the gradient ∇f .

- **Fastest Increase:** Occurs when we move in the direction of ∇f ($\theta = 0$).
- **Fastest Decrease:** Occurs when we move in the opposite direction of ∇f ($\theta = \pi$).
- **No Change:** Occurs when we move orthogonal to ∇f ($\theta = \pi/2$).

This is why we take a step in the direction of the **negative gradient** to go "downhill".



2.2 The Update Rule

The core update rule for Gradient Descent combines these ideas:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla f(\theta_{\text{old}}) \quad (2)$$

Where θ is the vector of our parameters (e.g., $\theta = [w_1, w_0]$) and α is the **learning rate**, which controls the size of our step.

3 Epochs, Iterations, and Batches Explained

Before diving into examples, it's crucial to understand these terms.

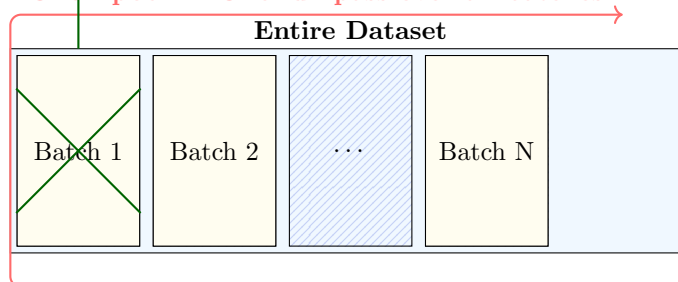
Epochs vs. Iterations

- **Epoch:** One full pass through the **entire** training dataset.
- **Batch:** A small subset of the training dataset.
- **Iteration:** A single update of the model's parameters. An iteration consists of processing one batch of data.

The relationship is: Number of Iterations per Epoch = (Total number of data points) / (Batch Size).

One Iteration = One parameter update

One Epoch = One full pass over all batches



4 Gradient Descent Variants: A Detailed Numerical Example

Problem Setup

- **Model:** $\hat{y} = w_1x + w_0$
- **Dataset:** $(1, 2), (2, 4), (3, 5), (4, 7)$
- **Initial Weights:** $w_1 = 1, w_0 = 0$
- **Learning Rate (α):** 0.01

4.1 1. Batch Gradient Descent (The Whole Dataset)

1 Epoch = 1 Iteration.

Iteration 1 (Full Epoch 1)**1. Calculate Gradients:**

$$\text{Point (1,2): } \hat{y} = 1, \text{Err} = 1 \implies \partial L/\partial w_1 = -2, \partial L/\partial w_0 = -2$$

$$\text{Point (2,4): } \hat{y} = 2, \text{Err} = 2 \implies \partial L/\partial w_1 = -8, \partial L/\partial w_0 = -4$$

$$\text{Point (3,5): } \hat{y} = 3, \text{Err} = 2 \implies \partial L/\partial w_1 = -12, \partial L/\partial w_0 = -4$$

$$\text{Point (4,7): } \hat{y} = 4, \text{Err} = 3 \implies \partial L/\partial w_1 = -24, \partial L/\partial w_0 = -6$$

2. Average Gradients:

$$\text{Avg } \partial L/\partial w_1 = (-2 - 8 - 12 - 24)/4 = -11.5$$

$$\text{Avg } \partial L/\partial w_0 = (-2 - 4 - 4 - 6)/4 = -4$$

3. Update Weights:

$$w_{1,\text{new}} = 1 - 0.01(-11.5) = \mathbf{1.115}$$

$$w_{0,\text{new}} = 0 - 0.01(-4) = \mathbf{0.04}$$

Iteration 2 (Full Epoch 2)

Using new weights $w_1 = 1.115, w_0 = 0.04$:

$$\text{Avg } \partial L/\partial w_1 = -9.575$$

$$\text{Avg } \partial L/\partial w_0 = -3.345$$

Update Weights:

$$w_{1,\text{new}} = 1.115 - 0.01(-9.575) = \mathbf{1.21075}$$

$$w_{0,\text{new}} = 0.04 - 0.01(-3.345) = \mathbf{0.07345}$$

4.2 2. Mini-Batch Gradient Descent (A Small Batch)

Batch size = 2. **1 Epoch = 2 Iterations.**

Epoch 1

Iteration 1 (Batch 1): Avg Gradients: $\partial L/\partial w_1 = -5, \partial L/\partial w_0 = -3$.

$$w_1 = 1 - 0.01(-5) = 1.05$$

$$w_0 = 0 - 0.01(-3) = 0.03$$

Iteration 2 (Batch 2): Avg Gradients: $\partial L/\partial w_1 = -16.54, \partial L/\partial w_0 = -4.59$.

$$w_1 = 1.05 - 0.01(-16.54) = \mathbf{1.2154}$$

$$w_0 = 0.03 - 0.01(-4.59) = \mathbf{0.0759}$$

Epoch 2

Iteration 3 (Batch 1): Avg Gradients: $\partial L/\partial w_1 = -3.71, \partial L/\partial w_0 = -2.21$.

$$w_1 = 1.2154 - 0.01(-3.71) = 1.2525$$

$$w_0 = 0.0759 - 0.01(-2.21) = 0.098$$

Iteration 4 (Batch 2): Avg Gradients: $\partial L/\partial w_1 = -11.00, \partial L/\partial w_0 = -3.04$.

$$w_1 = 1.2525 - 0.01(-11.00) = \mathbf{1.3625}$$

$$w_0 = 0.098 - 0.01(-3.04) = \mathbf{0.1284}$$

4.3 3. Stochastic Gradient Descent (A Single Point)

1 Epoch = 4 Iterations.

Epoch 1

Iteration 1 (Point 1,2): Gradients: $\partial L/\partial w_1 = -2, \partial L/\partial w_0 = -2$.

$$\begin{aligned}w_1 &= 1 - 0.01(-2) = 1.02 \\w_0 &= 0 - 0.01(-2) = 0.02\end{aligned}$$

Iteration 2 (Point 2,4): Gradients: $\partial L/\partial w_1 = -7.76, \partial L/\partial w_0 = -3.88$.

$$\begin{aligned}w_1 &= 1.02 - 0.01(-7.76) = 1.0976 \\w_0 &= 0.02 - 0.01(-3.88) = 0.0588\end{aligned}$$

Iteration 3 (Point 3,5): Gradients: $\partial L/\partial w_1 = -9.89, \partial L/\partial w_0 = -3.30$.

$$\begin{aligned}w_1 &= 1.0976 - 0.01(-9.89) = 1.1965 \\w_0 &= 0.0588 - 0.01(-3.30) = 0.0918\end{aligned}$$

Iteration 4 (Point 4,7): Gradients: $\partial L/\partial w_1 = -16.98, \partial L/\partial w_0 = -4.24$.

$$\begin{aligned}w_1 &= 1.1965 - 0.01(-16.98) = \mathbf{1.3663} \\w_0 &= 0.0918 - 0.01(-4.24) = \mathbf{0.1342}\end{aligned}$$

Epoch 2

Iteration 5 (Point 1,2): Gradients: $\partial L/\partial w_1 = -0.999, \partial L/\partial w_0 = -0.999$.

$$\begin{aligned}w_1 &= 1.3663 - 0.01(-0.999) = 1.3763 \\w_0 &= 0.1342 - 0.01(-0.999) = 0.1442\end{aligned}$$

Iteration 6 (Point 2,4): Gradients: $\partial L/\partial w_1 = -4.41, \partial L/\partial w_0 = -2.21$.

$$\begin{aligned}w_1 &= 1.3763 - 0.01(-4.41) = 1.4204 \\w_0 &= 0.1442 - 0.01(-2.21) = 0.1663\end{aligned}$$

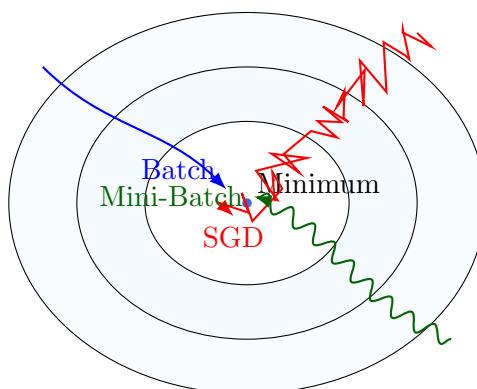
Iteration 7 (Point 3,5): Gradients: $\partial L/\partial w_1 = -3.44, \partial L/\partial w_0 = -1.15$.

$$\begin{aligned}w_1 &= 1.4204 - 0.01(-3.44) = 1.4548 \\w_0 &= 0.1663 - 0.01(-1.15) = 0.1778\end{aligned}$$

Iteration 8 (Point 4,7): Gradients: $\partial L/\partial w_1 = -8.02, \partial L/\partial w_0 = -2.01$.

$$\begin{aligned}w_1 &= 1.4548 - 0.01(-8.02) = \mathbf{1.5350} \\w_0 &= 0.1778 - 0.01(-2.01) = \mathbf{0.1979}\end{aligned}$$

4.4 Summary of Variants



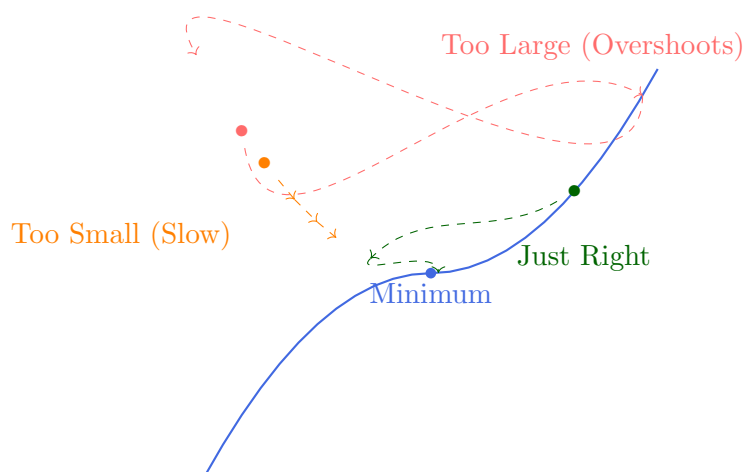
Visualizing Convergence Paths

5 The Learning Rate (α): The Goldilocks Problem

The learning rate is arguably the most important hyperparameter in Gradient Descent. Choosing the right value is like the Goldilocks story: you don't want it too big or too small, you want it "just right".

Analogy: Descending the Mountain with Different Step Sizes

- **Too Large (α is high):** You try to get down the mountain by taking giant leaps. You completely jump over the valley and might even land higher up on the other side. The process is unstable and never finds the bottom.
- **Too Small (α is low):** You take tiny, shuffling steps. You will eventually get to the bottom, but it will take an incredibly long time. The process is stable but very inefficient.
- **Just Right:** You take confident, appropriately sized steps, slowing down as the terrain flattens near the bottom. This is the ideal scenario.



A common strategy is to use a **learning rate schedule** (or decay), where the learning rate decreases over time. The update is applied at **every iteration**.

Learning Rate Decay Examples

Let's assume an initial learning rate $\alpha_0 = 0.1$. The variable t represents the **iteration number**.

Exponential Decay

This is an aggressive decay, good for when you are confident you are heading in the right direction early on.

- **Formula:** $\alpha_t = \alpha_0 e^{-kt}$ (Let's use decay rate $k = 0.5$)
- **Iteration 0:** $\alpha_0 = 0.1 \times e^0 = 0.1$
- **Iteration 1:** $\alpha_1 = 0.1 \times e^{-0.5} \approx 0.0607$
- **Iteration 2:** $\alpha_2 = 0.1 \times e^{-1} \approx 0.0368$

Step Decay

This is more controlled and predictable, allowing the model to "settle" at a certain learning rate before dropping again.

- **Concept:** Halve the learning rate every 100 iterations.
- **Iterations 0-99:** $\alpha = 0.1$
- **Iterations 100-199:** $\alpha = 0.05$